Angelia

Evidence-backed Windows investigation workbench

Technical project report
Version assessed: 0.4.0
Date: 25 October 2026
Status: Working case workflow and locally exercised release

A Windows investigation workbench that preserves source evidence, reconstructs process activity, records explainable findings, and exports a case report. This document describes the implementation, measured results, and practical scope.

Executive summary

Angelia is a local Windows investigation workbench. It collects live host activity or imports saved evidence into a case, preserves hashed source files, links events to processes where the source permits, generates cautious findings, and exports a self-contained HTML report. The same executable contains PE scoring, crash triage, and an experimental memory process inventory. Its intended use is to help an investigator answer what ran, what it touched, why an item was flagged, and which original record supports the conclusion. [S1, S2]

The current release has been built and exercised on Windows x64. All **29 configured tests passed**. A 30-second elevated live capture imported **428,276 ETW events** with **zero reported ETW losses and zero internal pool or queue drops**. The pre-collection process snapshot associated **426,947 events** with already-running processes; **211 events (0.05%)** remained without process identity. A controlled process that existed before collection contributed **3,273 snapshot-associated events**, including file creation and registry activity. Both retained sources passed SHA-256 verification. [E1, E2]

The package was extracted into a fresh local folder and its first-use demo completed without administrator rights: nine synthetic events were imported, three findings were generated, the source hash verified, and an HTML report exported. Report layout and search were checked in Edge at desktop and 390-pixel mobile width. This demonstrates local usability, not portability to every Windows installation. [E3]

The latest capture contained **64,214 unmapped event kinds**. The findings evaluation uses four controlled scenarios, and the memory parser and comparison workflow have synthetic test coverage. Angelia is an evidence-oriented investigation aid; its scores and crash classifications are leads for review. [E1, E4, E5]

Area	Current state	Evidence or limit
Case evidence and timeline	Implemented	Hashed source copies, SQLite case, source record IDs, searchable export [S1]
Elevated live collection	Field-checked locally	428,276 events; zero reported losses/drops [E1]
Initial process attribution	Field-checked locally	426,947 snapshot-associated; 211 unattributed [E1]
PE and behavior findings	Implemented, limited evaluation	Four controlled scenarios: TP 1, FP 1, FN 1, TN 1 [E4]
Crash triage in a case	Implemented and tested	Initial/final artifacts, input, binary, minidump, repeat comparison [E3]
Memory process inventory	Restricted x64 format	Parser and comparison checks pass on synthetic fixtures [E5]
Runtime package	Locally exercised	0.4.0 zip with demo and runtime DLLs [E3]

1. Problem, goals, and boundaries

Windows evidence often arrives in incompatible forms: live event streams, saved Event Log files, packet captures, executable files, and crash dumps. A raw event count or a single risk score does not tell another reviewer how a conclusion was reached. Angelia addresses that review problem by making a **case** the unit of work. A case records original inputs, normalizes what it can, keeps uncertain links explicit, and exposes the source behind each finding. [S1]

The product goals are:

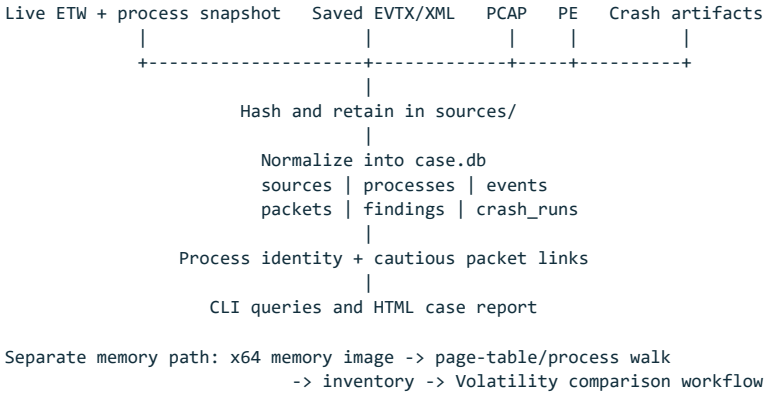
1. Preserve imported evidence as content-addressed files and record acquisition context.
2. Build a process-centered timeline from live ETW and saved Windows/Sysmon events.
3. Correlate network packets and processes only when identifiers and timing support a link.
4. Generate small, explainable findings with exact evidence and stated limitations.
5. Attach reproducible crash triage to the same case.
6. Export a report that a second reviewer can read without the source code.

The implemented scope is a **local Windows x64 tool**. Live ETW collection requires an elevated shell. Imports, case queries, report generation, and the package demo run unprivileged. Case commands are local and do not call an external service. The optional AI writeup is a separate action that sends the full crash artifact only when `--send` is specified. [S1, S2, E3]

`mem-analyze` is a separate command for a restricted x64 `PAGEDU64` format. Its output is a process inventory with explicit format errors and optional comparison against a separate live snapshot. [E5]

2. Architecture and case model

Angelia is a native C++20 executable with subcommands for collection, case work, memory inspection, PE scoring, crash triage, reporting, and evaluation. The collector writes a SQLite database; the case workbench imports this and other sources into a separate case database. The case directory holds `case.db`, `sources/`, optional incoming captures, and exported reports. [S1, S3]



The `sources` table records an input type, original path, retained path, SHA-256, import time, record count, and notes. A copied input is named by its hash; re-importing the same type and hash is idempotent. Event rows retain a source ID and source record number. `case verify` recomputes hashes and rejects missing, changed, or unsafe retained paths. These checks show whether retained bytes still match the import record; they do not establish that the original acquisition was trustworthy. [S1, S3]

The normalized `processes`, `events`, and `packets` tables support timeline queries. Findings retain rule ID, subject, severity, rationale, limitations, and structured evidence. Crash runs point to source IDs for the executable, input, initial capture, final classified artifact, and minidump. The HTML export embeds its own styling and script, so it opens without a service or remote assets. The export includes a source-integrity result at generation time. [S1, S3]

3. Acquisition, ingestion, and quality controls

3.1 Live Windows activity

The live collector enables process, image, file, and registry tracing. It materializes events into an internal queue and commits them to SQLite. Its summary reports committed events, event-type counts, throughput, maximum queue depth, commit latency, pool drops, queue drops, and ETW lost-event and lost-buffer counters. Microsoft documents ETW as a provider/controller/consumer system and exposes loss counters in trace-session statistics; zero reported loss is evidence about this run, not a guarantee that every activity was observable. [S3, E1, X1]

`case collect` now takes a live process snapshot immediately before ETW starts. This supplies PID, image, and creation time for processes already running when tracing begins. Their later events receive a `snapshot:<source>:<pid>:<creation-time>` key. A subsequently observed process-start record provides a newer key. Snapshot linkage is a time-based inference and remains labelled as such because missed starts or PID reuse could make it wrong. Events after an observed process end are not silently attached to the ended process. [S1, E1]

3.2 Saved Windows events

The workbench imports saved EVTX through the Windows Event Log API and accepts exported Event XML. Sysmon process GUIDs distinguish processes even when a numeric PID is reused; Sysmon records can also carry image hashes and network connections. Angelia preserves raw XML for less-specific events. Microsoft documents Sysmon's process GUID, optional network event, and hash fields. [S1, X2]

3.3 Packet capture

Classic PCAP is indexed into SQLite for supported Ethernet or raw IPv4 TCP/UDP packets. A packet is assigned to a process only when a Sysmon `NetworkConnect` event has the same protocol and endpoint tuple within two seconds. Reverse-direction packets can match. Different matching process identities produce `ambiguous`; no match produces `unlinked`. This is an investigative hint, not proof that the process sent a particular packet. PCAPNG, IPv6, and some fragment/protocol forms are not yet indexed. [S1]

3.4 Binary and crash inputs

`case add-pe` preserves a PE file and runs the static scorer. `case crash` executes a user-specified binary with an exact input or generated cyclic pattern, then runs capture, analysis, and classification. Only intentionally selected test programs should be executed. Crash evidence can contain input bytes and process memory; cases require private storage. [S1, E3]

4. Identity, correlation, and findings

Process identity quality depends on the input. Sysmon's `ProcessGuid` is the strongest supported case identifier. ETW process starts establish a PID-specific lifetime within an import. The initial snapshot provides an inferred identity for processes already running. If none of those sources safely identifies a process, Angelia leaves the event unattributed. This matters because joining all activity by PID alone would risk combining different processes after reuse. [S1, E1, X2]

The case finding rules currently flag: (a) a write to a Windows Run or RunOnce auto-start value, (b) an image load from a commonly user-writable location, and (c) at least 50 distinct file-write paths by one process. A finding includes the triggering source records, a rationale, and a limitation such as the possibility of a legitimate installer or backup. Rules require a process key, preventing unattributed events from being merged into a process-level accusation. PE findings carry the file hash, feature contribution, import/section evidence, and any matching Sysmon process hash. [S1, S3]

The PE scorer parses sections and imports, measures executable-section entropy, checks writable-and-executable sections and mitigation flags, and recognizes grouped import patterns such as process injection or crypto/enumeration/persistence shapes. Its score is a heuristic indicator. An import table or a high-entropy section alone is not evidence that the program performed malicious actions. [S4]

No machine-learning detector has been added on top of these rules. The available evaluation is too small to justify a general detection claim. The case report instead exposes exactly why a rule fired and what evidence is missing. [E4]

5. Crash triage and memory analysis

The crash pipeline uses Windows debugging to capture the first relevant exception and a companion minidump. The normalized artifact records exception code, fault address, registers, instruction and stack bytes, mitigation flags, input path, and dump path. The analyzer disassembles the faulting instruction with Zydis and correlates register or stack values with the cyclic input pattern. The deterministic classifier then assigns a provisional MSEC-style bucket and cites supporting evidence slugs. The case keeps the initial and classified artifacts separately so the classification can be replayed. [S5, E3]

Repeated crash runs are grouped by binary and input SHA-256. If buckets differ, the case marks disagreement rather than hiding it behind a single verdict. Agreement is not proof that the classification is correct. In the recorded controlled run, one unmitigated stack example was `Exploitable` with a saved-return-address pattern hit; nine heap runs across three variants were `Unknown` at 0.25 confidence. The `Unknown` result does not mean those programs are safe. [E3]

The memory path reads a restricted x64 Windows `PAGEDU64` image with a valid dump header and physical-memory runs. It uses the dump's page tables, resolves `_EPROCESS` offsets from a matching kernel PDB or explicit offsets, and walks `ActiveProcessLinks` to produce a process inventory. A separate live snapshot can be compared, but differences are

labelled `raw_only` or `snapshot_only` for review because the captures occur at different times. Unsupported formats fail clearly. A comparison script for Volatility 3 `windows.pslist` checks PID, image name, and process address on the same image. Parser and comparison mechanics were exercised with synthetic fixtures. [E5, X3, X4]

6. End-to-end operating workflow

1. Initialize a new case and choose a private storage location.
2. Collect live ETW as administrator or import saved EVTX/XML, an Angelia ETW database, PCAP, and/or PE evidence.
3. Query processes and timeline; examine uncertain or missing identity before interpreting activity.
4. Run `case analyze`, inspect findings with exact source records, then run `case verify`.
5. Optionally attach controlled crash triage for a selected executable and input.
6. Export `case report` for a second reviewer. Keep `case.db` and `sources/` if the reviewer must reproduce queries.

The shipped `demo.ps1` performs a safe first run using a synthetic event file. It creates a case, imports nine events, analyzes them, checks source integrity, and writes an HTML report. The demo deliberately contains both a false alarm and a missed scenario; it teaches how to read findings rather than demonstrating a field accuracy rate. [S2, E3]

7. Validation design and observed results

Validation uses four distinct evidence types: automated tests for code paths, synthetic load for queue and writer behavior, controlled scenarios with known expected events, and a real elevated local ETW capture. Each result is interpreted within its measured scope. [E1-E5]

Check	Method	Observed result	Interpretation
Build and automated suite	CMake/MSVC build; CTest	29/29 passed	Current code paths compile and pass configured tests [E3]
Collector load	One-million-event synthetic run	1,000,000 committed; zero internal pool/queue drops	Internal pipeline result only; not ETW field loss [E6]
Live capture	Elevated 30-second ETW run	428,276 imported; zero reported ETW losses/buffers and internal drops	Successful acquisition on this machine for this interval [E1]
Initial process attribution	Pre-existing controlled process	306 snapshot identities; 426,947 snapshot-associated events; 211 unattributed	Identity gap greatly reduced; links still inferred [E1]
Known workload	File and registry operations after trace start	PID 6656 had 3,273 snapshot-linked events and exact source records	Pre-existing process activity is reconstructable [E1]
Source integrity	Rehash retained evidence	2/2 live sources matched; test detected deliberate modification	Retained-byte integrity check works [E1, E3]
Rule evaluation	Four labelled synthetic scenarios	TP 1, FP 1, FN 1, TN 1	Demonstrates false alarm and miss; no field accuracy estimate [E4]
Crash integration	Controlled stack and repeated heap examples	Stack: one `Exploitable`; heap: nine `Unknown`	Pipeline and uncertainty recorded [E3]
First-use package	Fresh local zip extraction and demo	Nine events, three findings, verified source, exported report	Local first-use path works [E3]
Memory comparison	Synthetic parser/comparator tests	Passed	Supported mechanics behave as expected on fixtures [E5]

7.1 Live-capture interpretation

The snapshot-associated share was 426,947 / 428,276 = **99.69%**; only 211 / 428,276 = **0.05%** of events lacked process identity. The remaining 1,118 events had another established identity, such as an observed process start. The controlled helper started before tracing and produced file events such as source #2 record 47030 (`sample-0.txt` open), record 50094 (rename), registry record 51096 (key creation), and record 51480 (value set). Its 67 events just after `ProcessEnd` were left unattributed rather than guessed across a possible PID reuse. [E1]

An earlier 30-second capture before snapshot seeding imported 759,156 events but left 755,708 without process identity. The two captures had different workloads and rates, so this is an illustration of the fixed attribution gap, not a controlled performance comparison. The latest capture still contained **64,214** `Unknown` **event kinds**, about 15.0% of its events, chiefly unmapped Kernel-Process and Kernel-Registry IDs. Zero transport loss does not imply complete semantic coverage. [E1, E4]

7.2 Findings and older binary baseline

The four controlled finding scenarios yielded precision and recall of 0.5 **within that fixture only**. A legitimate installer writing a Run value was flagged; an encoded-command scenario without a covered indicator was missed. A real `notepad.exe` binary scored 0.000 with no static findings in one check. None of these observations estimates a population-level false-positive or detection rate. [E4]

An earlier, corrected 13-binary evaluation of the static/triage scorer reported TP 5, FP 0, TN 6, FN 2, and F1 0.833 at threshold 0.05. It used seven positive and six negative selected binaries and belongs to an older baseline. It is not comparable to the four-scenario case-rule test, and its precision is not evidence of real-world malware detection. [E6]

7.3 Report usability

The 0.4.0 HTML report includes evidence sources and hashes, an integrity result, coverage counters, findings with expandable evidence, crash detail, and a searchable timeline. A fresh extracted package generated the report without the build tree. In local Edge/Playwright checks, the title and content loaded, no page or console errors appeared, searching `notepad` reduced nine visible rows to one, and the 390-pixel layout had no page-level horizontal overflow. The report includes only its first 5,000 timeline rows; full-case queries remain available through the CLI. [E3, S1]

8. Security, privacy, and evidentiary interpretation

The case can contain personal file paths, command lines, event XML, executable files, crafted inputs, and process memory in crash minidumps. Store cases and exported reports in a private location. Hashing detects changes to retained bytes but is not encryption, acquisition authentication, or a complete chain of custody. A reviewer should record who collected each source, from which host, and when; the product currently does not enforce those procedural controls. [S1, E3]

The optional AI writeup command is outside the normal case workflow. `--dry-run` prints the full prompt locally; `--send` explicitly transmits the full crash artifact JSON to the configured service. Because that JSON can carry sensitive paths and state, it should be reviewed before transmission. The deterministic verdict is provisional even when an AI-generated explanation is added. [S2, E3]

Several results are leads rather than proof: Run keys can be used by installers; high file-write counts can arise from backups or builds; packet-process associations are based on tuple and time, not a unique packet ownership record; a process seen only in one of two memory observations may simply have started or exited between captures. Findings and inventories expose these limitations so a reviewer can ask for additional evidence. [S1, E5]

9. Practical limits and next work

Event coverage and detection quality: map or intentionally classify the high-volume unknown ETW IDs and expand the set of normal and controlled suspicious scenarios. The live run's zero behavior findings is appropriate for the benign workload; the case rules remain review aids. [E1, E4]

Usability and robustness: remove or paginate the HTML report's 5,000-row display cap, add a timeout to crash capture for programs that hang, and consider PCAPNG/IPv6 ingestion. [S1]

Scope control: keep memory and any future network expansions evidence-based. New parsers or scoring rules should have a supported input definition and a clear failure or uncertainty state before appearing as definitive report conclusions.

10. Conclusion

Angelia now provides a working local investigation workflow: source-preserving case import, live acquisition with visible loss counters, process-aware timeline, cautious packet association, explainable findings, crash triage, evidence verification, and a portable HTML export. A current elevated capture shows that initial process snapshots materially improve attribution for already-running processes on the tested machine. The release package can be used locally without reading the source code. The project is useful today for controlled Windows investigation and review, with memory and detection outputs interpreted within their documented scope. [E1, E3-E5]

Appendix A. Reproduction commands

Run these from the repository root in PowerShell. Use a private case path. `case collect` needs an elevated shell.

```
$a = '.\build\default\bin\angelia.exe'
& $a --version
& $a case init .\cases\my-case
& $a case collect .\cases\my-case --duration 30
& $a case import-evtx .\cases\my-case C:\Evidence\Sysmon.evtx
& $a case import-pcap .\cases\my-case C:\Evidence\traffic.pcap
& $a case analyze .\cases\my-case
& $a case findings .\cases\my-case
& $a case verify .\cases\my-case
& $a case report .\cases\my-case --out .\cases\my-case\report.html
```

For controlled crash triage, use an intentionally selected program and input:

```
& $a case crash .\cases\my-case --binary C:\Lab\program.exe --input C:\Lab\input.bin --repeat 3
& $a case crashes .\cases\my-case
```

For the packaged first-use demo, extract `dist/angelia-0.4.0.zip` and run `powershell -NoProfile -ExecutionPolicy Bypass -File .\demo.ps1`. For the exact live validation case, see [E1] and query PID 6656; the HTML export intentionally shows only the first 5,000 rows.

Appendix B. Evidence and source index

Project sources

- [S1] Case design and commands; case workbench implementation.
- [S2] Quick-start guide; README; release package.
- [S3] Case schema and report implementation; collector sources.
- [S4] Static PE scorer.
- [S5] Crash artifact schema; classifier.

Recorded results

- [E1] Elevated snapshot validation and the retained local case sources.
- [E2] Case release validation and the ETW source statistics in the case database.
- [E3] Release validation; crash smoke test; case smoke test.
- [E4] Findings validation and its labeled fixture.
- [E5] Memory validation status and method; comparison code.
- [E6] Baseline and 13-binary summary.

External technical references

- [X1] Microsoft, About Event Tracing and EVENT_TRACE_PROPERTIES (loss counters).
- [X2] Microsoft Sysinternals, Sysmon overview and event fields.
- [X3] Microsoft, Generate live kernel dumps using Task Manager.
- [X4] Volatility Foundation, Volatility 3 Windows tutorial and windows.pslist source.

Remaining validation: compare two genuine supported Windows memory images with Volatility 3 and run the release zip on a separate clean Windows installation. [E3, E5]